

Microsoft SharePoint “ToolShell” Incident:

Unauthenticated Remote Code Execution via Insecure Deserialization
(CVE-2025-53770 / CVE-2025-53771)

Hyunseo Choi

College of Engineering

Hanyang University

Dept. of Information Systems

Seoul, Korea

Junho Uh

College of Engineering

Hanyang University

Dept. of Information Systems

Seoul, Korea

Yewon Joung

College of Engineering

Hanyang University

Dept. of Information Systems

Seoul, Korea

Jeongo Heo

College of Engineering

Hanyang University

Dept. of Information Systems

Seoul, Korea

Abstract—This survey investigates the 2025 Microsoft SharePoint “ToolShell” incident, a chain of vulnerabilities affecting on-premises SharePoint Server that enabled unauthenticated remote code execution (RCE). The analysis centers on CVE-2025-53770, a critical deserialization-of-untrusted-data flaw (CWE-502) rated 9.8 on the CVSS v3.1 scale, and its companion authentication-bypass variant CVE-2025-53771. We show that these vulnerabilities did not arise in isolation but emerged as bypasses of two previously patched flaws, CVE-2025-49704 and CVE-2025-49706, illustrating how incomplete fixes can be re-weaponized. We reconstruct the end-to-end attack chain—from exploitation of the `ToolPane.aspx` endpoint and a forged `SignOut.aspx` referer, through deployment of the `spinstall0.aspx` payload, to theft of ASP.NET MachineKey secrets and forgery of signed ViewState payloads enabling persistence and follow-on ransomware. Drawing on public advisories from Microsoft, CISA, and major threat-intelligence vendors, we consolidate indicators of compromise, Sigma-based detection rules, defensive log-scanning code, a safe emulation plan, and a MITRE ATT&CK mapping. We conclude that while the exploit is product-specific, its underlying lessons—avoiding unsafe deserialization, protecting signing keys, rotating secrets after compromise, and reducing internet exposure—generalize broadly across enterprise web applications.

Index Terms—SharePoint, ToolShell, CVE-2025-53770, insecure deserialization, remote code execution, web shell, MITRE ATT&CK, incident analysis

I. INTRODUCTION

Enterprise collaboration platforms occupy a uniquely dangerous position in the modern attack surface: they are widely deployed, deeply integrated with internal identity and document stores, and frequently exposed to the internet. A single server-side flaw in such a platform can therefore convert routine internet exposure into full enterprise compromise. The 2025 Microsoft SharePoint “ToolShell” incident is a vivid illustration of this risk, and it is the subject of this survey.

What makes ToolShell instructive is the way several distinct failures compounded. Rather than a lone bug, it was a chain: an authentication-bypass flaw and a code-injection flaw were combined, patched, and then revived through bypass variants that defeated the initial fixes. Once code execution was achieved, attackers did not stop at a transient foothold—they stole the server’s cryptographic signing material and used it to forge trusted payloads, establishing durable access that a naive patch could not revoke. This progression, from public

disclosure to in-the-wild exploitation to ransomware follow-up, unfolded over a matter of days, making the incident an unusually well-documented case for study.

This paper makes three contributions. First, we reconstruct the end-to-end attack chain and clarify the often-confused relationships among the four associated CVEs. Second, we consolidate scattered public reporting—from vendor advisories, national cyber agencies, and threat-intelligence teams—into a single defensive reference comprising indicators of compromise, Sigma detection rules, and a defensive log-scanning tool. Third, we generalize the case beyond SharePoint, arguing that its root weakness and defensive lessons apply broadly to enterprise web applications. The remainder of the paper is organized accordingly: incident background, technical analysis, detection and defensive tooling, a safe emulation plan, mitigation guidance, and a discussion of generalizability.

II. INCIDENT BACKGROUND

Microsoft SharePoint is widely used in enterprise environments for document management, collaboration, workflow management, and integration with Microsoft services. Because many organizations expose SharePoint servers to the internet, a vulnerability in SharePoint can become a high-impact enterprise entry point.

The ToolShell incident affected *on-premises* SharePoint Server, not SharePoint Online in Microsoft 365. Microsoft explicitly stated that the vulnerabilities apply to on-premises SharePoint servers only and that SharePoint Online is not impacted [1]. Unit 42 reported that the ToolShell vulnerability set included CVE-2025-49704, CVE-2025-49706, CVE-2025-53770, and CVE-2025-53771, and that these vulnerabilities enabled attackers to achieve remote code execution without valid credentials [2].

III. TIMELINE OF THE INCIDENT

Table I summarizes the publicly reported timeline, from the original Pwn2Own demonstration of the related flaws through emergency patching and post-incident malware analysis.

IV. TECHNICAL ANALYSIS

A. Vulnerability Type

The root cause is unsafe deserialization. In general, deserialization converts stored or transmitted data back into

TABLE I: Reported Timeline of the SharePoint ToolShell Incident

Date	Event
May 2025	Related SharePoint flaws demonstrated at Pwn2Own Berlin (per later vendor reporting).
Jul 8, 2025	Microsoft patches earlier related flaws CVE-2025-49704 and CVE-2025-49706.
Jul 17–18	Scanning and exploitation observed in the wild; Unit 42 reports activity from Jul 17 [2].
Jul 19	Microsoft publishes customer guidance for CVE-2025-53770 [1].
Jul 20	CISA adds CVE-2025-53770 to its Known Exploited Vulnerabilities catalog [3].
Jul 21	Emergency patches released for SharePoint Subscription Edition, 2019, and 2016.
Aug 6	CISA releases a Malware Analysis Report on associated malicious files.

program objects. When an application deserializes attacker-controlled data without strict validation, the attacker may cause it to instantiate dangerous objects or execute unintended logic. CVE-2025-53770 allows an unauthorized attacker to execute code over the network against vulnerable on-premises SharePoint Server systems; NVD maps the flaw to CWE-502 [4].

A key nuance is the relationship among the four CVEs. CVE-2025-49706 is an authentication-bypass (spoofing) flaw, while CVE-2025-49704 is a code-injection flaw enabling RCE. The two were chained at Pwn2Own to bypass authentication and then execute code. CVE-2025-53770 and CVE-2025-53771 subsequently emerged as bypasses of the incomplete July 8 fixes, demonstrating how partial patches can be re-weaponized.

B. Attack Chain Overview

At a high level, the attack chain proceeds as follows:

- 1) Identify an internet-facing on-premises SharePoint server.
- 2) Send malicious requests targeting vulnerable SharePoint functionality.
- 3) Bypass authentication by abusing exposed processing behavior.
- 4) Upload or trigger a malicious ASPX payload.
- 5) Extract cryptographic secrets such as ASP.NET MachineKey values.
- 6) Use the stolen keys to forge valid signed (ViewState) payloads and maintain access.
- 7) Perform post-exploitation: reconnaissance, lateral movement, data theft, or ransomware deployment.

Trend Micro reported that attackers targeted the SharePoint ToolPane.aspx endpoint and used a specific Referer value to bypass authentication, resulting in malicious ASPX file deployment [5]. The observed file spinstall0.aspx was used to extract cryptographic secrets from the SharePoint instance. Because the stolen MachineKey signs ViewState, the attacker can thereafter craft trusted payloads that SharePoint deserializes and executes—turning a one-time exploit into durable, signed access.

TABLE II: Selected Indicators of Compromise

Type	Indicator
Suspicious file SHA-256	...\LAYOUTS\spinstall0.aspx 92bb4ddb98eeaf11fc15bb32e71d0a6- 3256a0ed826a03ba293ce3a8bf057a514
IIS pattern	POST /_layouts/15/ToolPane.aspx? DisplayMode=Edit&a=/ToolPane.aspx
Referer	/_layouts/SignOut.aspx
IP examples	107.191.58.76, 104.238.159.149, 96.9.125.147

V. INDICATORS OF COMPROMISE

The Canadian Centre for Cyber Security published several useful detection indicators [6], summarized in Table II. Key signals include the presence of spinstall0.aspx, suspicious POST requests to the SharePoint ToolPane endpoint carrying a SignOut referer, and traffic from known suspicious IP addresses.

IOC-based detection should not be the only control, because attackers can change filenames, IP addresses, payload names, and infrastructure. Behavior-based detection is more reliable—in particular, monitoring unexpected ASPX creation, suspicious w3wp.exe child processes, PowerShell execution, and access to SharePoint configuration secrets.

VI. DETECTION RULE SET

We developed three Sigma rules covering the main stages of the attack: (i) the suspicious IIS request pattern, (ii) malicious ASPX file creation in SharePoint layout directories, and (iii) IIS worker processes spawning suspicious child processes. The full rule set is available in our repository [7]; Listing 1 shows the representative initial-access rule, and Table III summarizes all three.

Listing 1: Sigma rule: suspicious IIS request (initial access).

```

title: SharePoint ToolShell Suspicious IIS Request
status: experimental
logsource:
  category: webserver
detection:
  selection_method:
    cs-method: 'POST'
  selection_uri:
    cs-uri-stem|contains: '/_layouts/15/ToolPane.aspx'
    cs-uri-query|contains: 'DisplayMode=Edit&a=/ToolPane.
    ↳ aspx'
  selection_referer:
    cs-referer|contains: '/_layouts/SignOut.aspx'
  condition: selection_method and selection_uri
    and selection_referer
level: high
tags: [attack.initial_access, attack.t1190, cve
    ↳ .2025-53770]

```

VII. DEFENSIVE SOURCE CODE

We provide defensive tooling: a Python scanner that parses W3C-format IIS logs for ToolShell-like indicators. The full scanner is available in our repository [7]; Listing 2 shows its core matching logic.

TABLE III: Summary of Sigma Detection Rules

Detection target	ATT&CK tag	Level
Suspicious IIS request	T1190	high
ASPX file creation in LAYOUTS	T1505.003	critical
w3wp.exe spawns shell	T1059, T1105	high

TABLE IV: MITRE ATT&CK Mapping

Attack stage	Technique
Exploiting internet-facing SharePoint	T1190
Web shell deployment	T1505.003
Command/PowerShell execution	T1059
Tool transfer	T1105
Secret/key theft	T1552
Lateral movement	T1021
Ransomware impact	T1486

Listing 2: Core detection logic of the defensive IIS log scanner.

```
suspicious_post = (
    method == POST
    and /_layouts/15/toolpane.aspx in stem
    and displaymode=edit&a=/toolpane.aspx in query
    and /_layouts/signout.aspx in referer)

suspicious_webshell = (
    method in {GET, POST}
    and /_layouts/15/spinstall0.aspx in stem)

if suspicious_post or suspicious_webshell:
    alert(path, line_number, source_ip, method)
```

The scanner reads each log line, reconstructs fields from the #Fields: header, and flags either the malicious POST pattern or direct access to the spinstall0.aspx web shell. It is intentionally read-only and signature-light, intended to complement—not replace—the behavior-based detection discussed in Section VIII.

VIII. SAFE EMULATION PLAN

The goal of emulation is to test detection and response. AttackIQ published an emulation-oriented assessment focused on validating security controls against the relevant tactics, techniques, and procedures [8]. Our plan emulates each stage of the attack using only synthetic or isolated artifacts: synthetic W3C IIS loglines for initial access, a harmless test file in an isolated lab path for persistence, controlled telemetry replay for execution, and simulated log/event generation for the attempted MachineKey access (no real keys are touched). Response and recovery steps—key rotation, IIS restart, patch verification, and IOC hunting—are validated in a staging environment. Table IV maps the emulated behaviors to MITRE ATT&CK.

IX. MITIGATION AND SOLUTION GUIDELINE

Microsoft recommended that customers use supported on-premises SharePoint versions, apply the latest security updates, deploy Microsoft Defender for Endpoint or equivalent protection, enable and correctly configure AMSI, and rotate

TABLE V: Prioritized Mitigation Actions

#	Action	Reason
1	Apply SharePoint security updates	Removes vulnerable code path
2	Rotate ASP.NET MachineKey	Stops abuse of stolen keys
3	Restart IIS after rotation	Activates new configuration
4	Enable SharePoint AMSI	Adds script/malware inspection
5	Deploy EDR on servers	Detects web shell, lateral movement
6	Remove internet exposure	Reduces attack surface
7	Hunt ASPX in LAYOUTS dirs	Finds web-shell artifacts
8	Review IIS logs for ToolPane	Finds exploitation attempts
9	Rotate service credentials	Limits impact of stolen config
10	Run IR if exposed	Patching alone may be insufficient

TABLE VI: Generalization of the ToolShell Pattern

Component	Generalized risk
Unsafe deserialization	Any app deserializing untrusted input
Secret extraction	JWT keys, API keys, signing/session secrets
Web shell	JSP, PHP, ASPX, Node.js server-side payloads
Patch bypass	Incomplete patching in complex software
Persistence	Forged tokens, sessions, signed payloads
Follow-on attack	Data theft, lateral movement, ransomware

SharePoint Server ASP.NET machine keys [1]. A prioritized action list is given in Table V.

The Cyber Security Agency of Singapore similarly warned that any organization with an internet-exposed SharePoint server during the exploitation window should treat it as at risk, and that patching alone is insufficient if the server was already compromised; it emphasized key rotation, IIS restart, and removal of attacker artifacts such as web shells [9].

X. APPLICABILITY AND EXTENSION

CVE-2025-53770 is specific to Microsoft SharePoint Server on-premises environments and, per Microsoft, does not affect SharePoint Online. However, the underlying vulnerability *pattern* is broadly applicable. The core weakness—unsafe deserialization—appears across .NET, Java, PHP, Python, and Node.js applications, and the broader attack pattern recurs in other enterprise systems, as summarized in Table VI.

Thus the exploit is product-specific, but the lesson is general: do not deserialize untrusted data, protect signing keys, rotate secrets after compromise, reduce internet exposure, and monitor web-server behavior.

XI. CONCLUSION

The Microsoft SharePoint ToolShell incident is a strong recent case study because it combines multiple modern security issues—unsafe deserialization, authentication bypass, patch bypass, web-shell deployment, cryptographic key theft, and ransomware follow-up. Unlike a simple vulnerability report, it demonstrates why patching alone may not be sufficient after exploitation: once MachineKey material is stolen, an attacker can forge signed payloads that survive a naive

patch. A complete defense therefore combines patching, key rotation, IIS restart, web-shell hunting, IIS log analysis, EDR deployment, and removal of public exposure. From a survey perspective, the case is valuable because it is backed by public advisories, IOC data, Sigma-style detection, safe emulation paths, and lessons that extend cleanly to other enterprise web applications.

REFERENCES

- [1] Microsoft Security Response Center, “Customer Guidance for SharePoint Vulnerability CVE-2025-53770,” <https://msrc.microsoft.com/blog/2025/07/customer-guidance-for-sharepoint-vulnerability-cve-2025-53770/>, 2025.
- [2] Unit 42, Palo Alto Networks, “Active Exploitation of Microsoft SharePoint Vulnerabilities: Threat Brief,” <https://unit42.paloaltonetworks.com/microsoft-sharepoint-cve-2025-49704-cve-2025-49706-cve-2025-53770/>, 2025.
- [3] Cybersecurity and Infrastructure Security Agency, “CISA Adds One Known Exploited Vulnerability, CVE-2025-53770 “ToolShell,” to Catalog,” <https://www.cisa.gov/news-events/alerts/2025/07/20/cisa-adds-one-known-exploited-vulnerability-cve-2025-53770-toolshell-catalog>, 2025.
- [4] NIST National Vulnerability Database, “CVE-2025-53770 Detail,” <https://nvd.nist.gov/vuln/detail/CVE-2025-53770>, 2025.
- [5] Trend Micro, “Proactive Security for CVE-2025-53770 and CVE-2025-53771 SharePoint Attacks,” https://www.trendmicro.com/en_us/research/25/g/cve-2025-53770-and-cve-2025-53771-sharepoint-attacks.html, 2025.
- [6] Canadian Centre for Cyber Security, “AL25-009 Vulnerability Impacting Microsoft SharePoint Server,” <https://www.cyber.gc.ca/en/alerts-advisories/al25-009-vulnerability-impacting-microsoft-sharepoint-server-cve-2025-53770>, 2025.
- [7] Project Authors, “Information Security Final Project: SharePoint Tool-Shell Detection Rules and Defensive Tooling,” https://github.com/pupwchk/infosec_final, 2026.
- [8] AttackIQ, “Emulation Assessment for Microsoft SharePoint ToolShell Vulnerabilities,” <https://www.attackiq.com/>, 2025.
- [9] Cyber Security Agency of Singapore, “Active Exploitation of Microsoft SharePoint Server Vulnerabilities,” <https://www.csa.gov.sg/alerts-advisories/>, 2025.